

Obstacles for a Llama

Een lama wil reizen door het Hoogland van de Andes. Hij heeft een kaart van het hoogland, een rooster van $N \times M$ vierkante cellen. De rijen van de kaart zijn genummerd van 0 tot en met $N - 1$ van boven naar beneden, en de kolommen zijn genummerd van 0 tot en met $M - 1$ van links naar rechts. De cel van de kaart in rij i en kolom j ($0 \leq i < N, 0 \leq j < M$) wordt genoteerd met (i, j) .

De lama heeft het klimaat van het hoogland bestudeerd en is erachter gekomen dat alle cellen in elke rij van de kaart dezelfde **temperatuur** hebben en alle cellen in elke kolom van de kaart dezelfde **vochtigheid**. De lama heeft je twee arrays met gehele getallen T en H met respectievelijk lengte N en M gegeven. Hier is $T[i]$ ($0 \leq i < N$) de temperatuur van de cellen in rij i , en $H[j]$ ($0 \leq j < M$) de vochtigheid van de cellen in kolom j .

De lama heeft ook de flora van het hoogland bestudeerd en heeft opgemerkt dat (i, j) **vrij van vegetatie** is dan en slechts dan als de temperatuur groter is dan de vochtigheid, formeel $T[i] > H[j]$.

De lama kan alleen over het hoogland reizen door **valide paden** te volgen. Een valide pad is een reeks van verschillende cellen die aan de volgende voorwaarden voldoen:

- Elk paar opeenvolgende cellen in het pad delen een gemeenschappelijke zijde.
- Alle cellen in het pad zijn vrij van vegetatie.

Jouw taak is om Q vragen te beantwoorden. Voor elke vraag krijg je vier gehele getallen: L, R, S , en D . Je moet bepalen of er een valide pad bestaat zodat:

- Het pad begint bij $(0, S)$ en eindigt bij $(0, D)$.
- Alle cellen van het pad in kolommen L tot en met R liggen

Het is gegarandeerd dat $(0, S)$ and $(0, D)$ vrij zijn van vegetatie.

Implementatiedetails

De eerste procedure die je moet implementeren is:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : een array van lengte N , de temperatuur in elke rij.
- H : een array van lengte M , de vochtigheid in elke kolom.

- Deze procedure wordt precies één keer aangeroepen bij elk testgeval, voor de aanroepen naar `can_reach`.

De tweede procedure die je moet implementeren is:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : gehele getallen die een vraag beschrijven.
- Deze procedure wordt Q keer aangeroepen bij elk testgeval.

Deze procedure moet `true` teruggeven dan en slechts dan als er een valide pad bestaat van cel $(0, S)$ naar cel $(0, D)$, zodat elke cel in het pad ligt binnen de kolommen L tot en met R .

Beperkingen

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ voor elke i zodat $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ voor elke j zodat $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- De cellen $(0, S)$ en $(0, D)$ zijn allebei vrij van vegetatie.

Subtaken

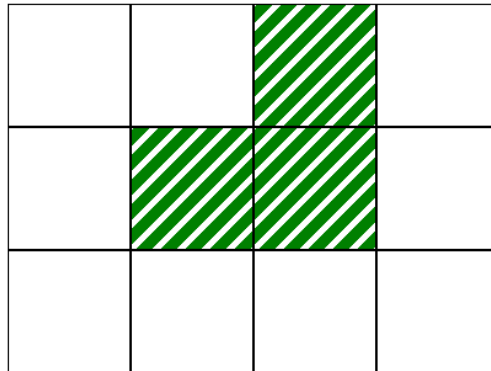
Subtaak	Score	Extra beperkingen
1	10	$L = 0, R = M - 1$ voor elke vraag. $N = 1$.
2	14	$L = 0, R = M - 1$ voor elke vraag. $T[i - 1] \leq T[i]$ voor elke i zodat $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ voor elke vraag. $N = 3$ en $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ voor elke vraag. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ voor elke vraag.
6	17	Geen extra beperkingen.

Voorbeeld

Beschouw de volgende aanroep.

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

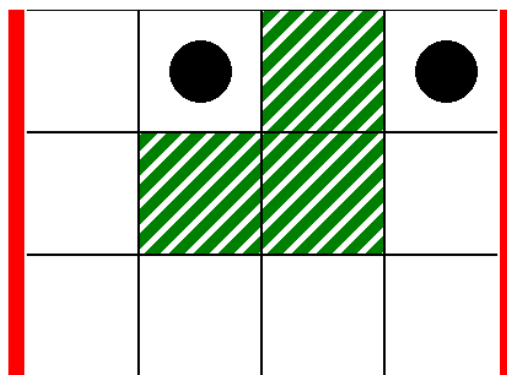
Dit correspondeert met de kaart in de volgende afbeelding, waar witte cellen vrij zijn van vegetatie:



Bekijk als eerste vraag de volgende aanroep:

```
can_reach(0, 3, 1, 3)
```

Dit correspondeert met de situatie in de volgende afbeelding, waar de dikke verticale lijnen de kolommen van $L = 0$ tot en met $R = 3$ aangeven, en de zwarte cirkels de begin- en eindcellen.



In dit geval kan de lama vanaf cel $(0,1)$ cel $(0,3)$ bereiken via het volgende valide pad:

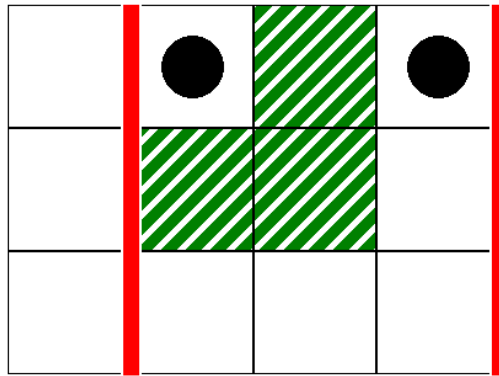
$(0,1), (0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (1,3), (0,3)$

Daarom moet deze aanroep `true` teruggeven.

Bekijk als tweede vraag de volgende aanroep:

```
can_reach(1, 3, 1, 3)
```

Dit correspondeert met de volgende afbeelding:



In dit geval is er geen valide pad van cel $(0,1)$ naar cel $(0,3)$ zodat alle cellen in het pad binnen kolommen 1 tot en met 3 liggen. Daarom moet deze aanroep `false` teruggeven.

Voorbeeld Grader

Input format:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Hier zijn $L[k]$, $R[k]$, $S[k]$ and $D[k]$ ($0 \leq k < Q$) de parameters voor elke aanroep naar `can_reach`.

Output Format:

```
A[0]
A[1]
...
A[Q-1]
```

Hier is $A[k]$ ($0 \leq k < Q$) 1 als `can_reach(L[k], R[k], S[k], D[k])` `true` heeft teruggegeven, en anders 0.